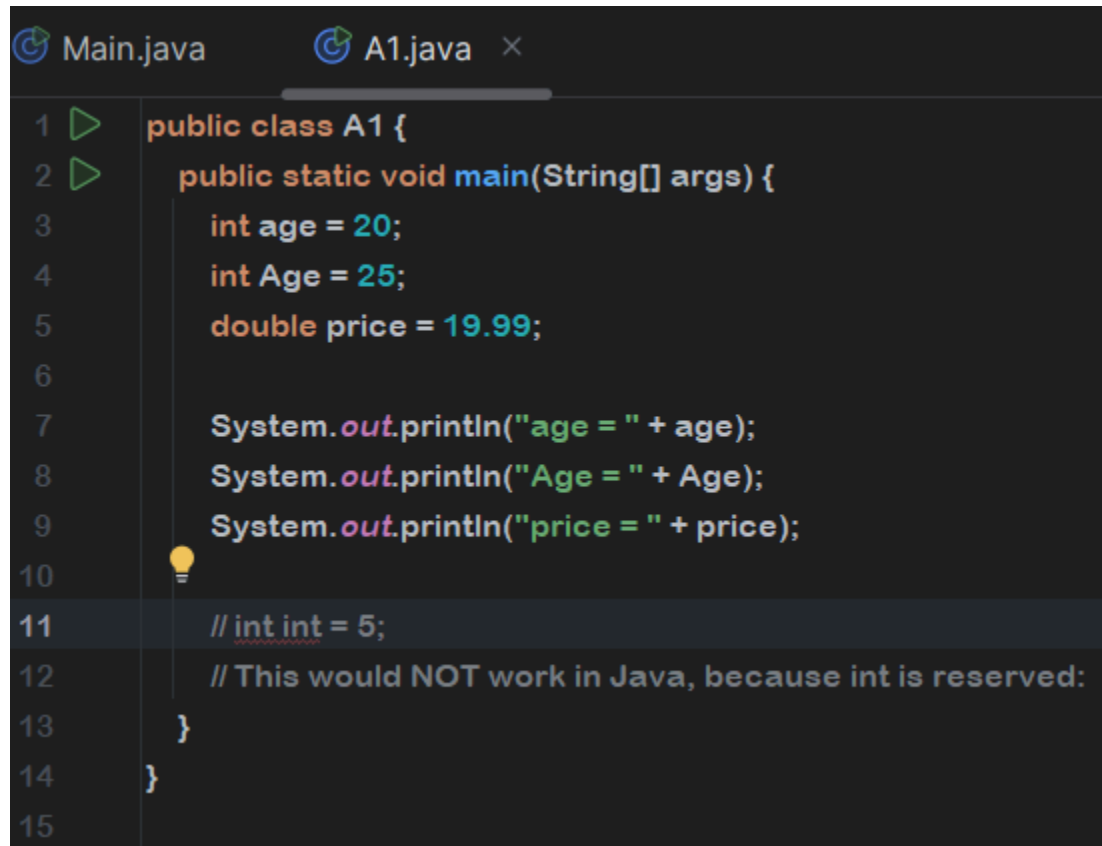


Duy Hoang
CSIT 313
HW3

A1.) Both Java and Python are case-sensitive, meaning that uppercase and lowercase letters are distinguishable. Many C-based languages are case sensitive, which Java is covered in. Java uses reserved words for the definition of primitive data types words like int, float, and double for real numbers are all reserved in Java and as such, they can not be used as names of variables. However, because the variables are not declared with a type keyword first, unlike in Java, the rules are not as stringent in Python. Designed for high readability, Python uses dynamic type binding, therefore, without manually specifying the type of the data, a variable in Python gets its type when a value is assigned to it. int and float serve as the built-in name of a type rather than a type-declaration keyword in Java. Java is more stringent while Python more at ease due to another difference between Java being statically typed and Python variables being dynamically typed. Once a variable is declared as an int or double its type stays fixed in Java while in Python a variable can change during execution.

Java



```
1  public class A1 {
2      public static void main(String[] args) {
3          int age = 20;
4          int Age = 25;
5          double price = 19.99;
6
7          System.out.println("age = " + age);
8          System.out.println("Age = " + Age);
9          System.out.println("price = " + price);
10
11         // int int = 5;
12         // This would NOT work in Java, because int is reserved:
13     }
14 }
15
```

```
age = 20
Age = 25
price = 19.99
```

This shows that Java is case sensitive, variables must be declared with a type, and int and double are type words in Java.

Python

```
A1.py ×
1  age = 20
2  Age = 25
3  price = 19.99
4
5  print("age =", age)
6  print("Age =", Age)
7  print("price =", price)
8
9  x = 10
10 print("x =", x, "type =", type(x))
11
12 x = 10.5
13 print("x =", x, "type =", type(x))
14
15 x = "hello"
16 print("x =", x, "type =", type(x))
17
18 # In Python, this is allowed, although it is bad practice:
19 int = 7
20 print("int =", int)
```

```
"C:\Users\Duy Hoang\PycharmProje
age = 20
Age = 25
price = 19.99
x = 10 type = <class 'int'>
x = 10.5 type = <class 'float'>
x = hello type = <class 'str'>
int = 7
```

This shows that Python is case sensitive, does not require a type declaration, and a variable can change type.

A2.) With static scoping, sub1 prints 5. Static scoping decides depending on the lexical structure of the program which variable is used. As sub1 is defined in the global scope, it uses the global variable x which is 5.

With dynamic scoping, sub1 prints 10. Dynamic scoping decides based on which functions are called in which order. As sub1 is then called from sub2, it uses the x local variable in sub2 which is 10.

A3.)

Point 1

- a - from definition 1
- b - from definition 2
- c - from definition 2
- d - from definition 2

Point 2

- a - from definition 1
- b - from definition 2
- c - from definition 3
- d - from definition 3
- e - from definition 3

Point 3

- a - from definition 1
- b - from definition 2
- c - from definition 2
- d - from definition 2

Point 4

- a - from definition 1
- b - from definition 1
- c - from definition 1

Part B

Scope is a concept in programming languages that determines where in a program a variable can be accessed. Scope is crucial for both ensuring correctness and helping programmers manage variables in large software projects. This simulation explores the scope of variables defined in a for loop in C, C++, and Java. The focus is on whether a variable declared in the initialization part of a for statement remains visible after the for loop terminates its execution.

To probe this behavior, simple programs are coded up for all three languages. In each program the variable `i` is declared within the initialization statement of the for loop:

```
for (int i = 0; i < 3; i++)
```

The program then prints the value of `i` inside the loop to show that the variable is available inside the loop, and then tries to print the value of `i` outside the loop at the end. If they are visible the program compiles and executes, otherwise the name error is issued by the compiler.

The results of running those programs show that in all three cases the variable `i` declared in the for loop is not seen after the loop exits: the compiler complains that the variable is not defined when `i` is printed outside the loop. The underlying cause of this result is the concept of block scope, a widely used feature in modern programming languages. As defined by Robert W. Sebesta in his textbook *Concepts of Programming Languages*, a block is a piece of code enclosed in curly braces `{ }` and the variables declared within the block are visible only within the block itself. This helps increase program safety and readability by avoiding unintended access and updates of variables. In C, C++, and Java, the body of a loop counts as a block and the variables declared in the header of the loop are assumed to be part of the block containing the loop.

Even if C, C++, and Java are very different languages, they all rely on the same scoping rules as they belong to the family of statically scoped programming languages. Static scoping dictates whether a variable is visible in a given piece of code based on the text of the program rather than on the order of function calls at runtime. The variable `i` is defined in the bracketed structure of the loop, thus the compiler limits its scope to the block containing the loop.

We can then conclude that C, C++, and Java treat the variables defined in the initialization statement of the for in the same way: they are visible only while the loop is executing. This behavior is characteristic of block scope rules and gives an example of how modern programming languages restrict variable access.